CS 301R · KEEP IT BESIDE THE KEYBOARD

The Debugging Loop

Method beats knowledge: evidence before belief.

0. **Say the failure exactly, and reproduce it.** Expected vs. actual, with the details ("*every* row, *the same* date"). Find the steps that make it happen every time.
1. **Hypothesize: two or three, not one.** Each names *where* the defect could be: the data, the display, the query, the config, the input.
2. **Test the cheapest one first,** not the one you believe. Vary the input · look at the stored data · add one log line where two guesses split · the debugger · `git log` / `git diff` / `git bisect` · shrink the case.
3. **Review.** Dead? Cross it out and say so: that's progress. Standing? Narrow until you can **point to the defect**, **explain when it bites**, and **propose a fix**.
4. **Fix and verify.** A failing test first (and ask why the old tests missed it) · fix the defect, not the symptom · run everything · walk the feature: did the fix break it? · remove your log lines.

You see the **failure** → the program hits a **fault** → caused by a **defect** in the code → from an **error**. Investigate backward.

The debug log: three lines

Saw	the failure, precisely
Suspected	each hypothesis, wrong ones too, and the evidence that settled it
Was	the defect, the fix, the test that now covers it

In the **bug's issue**, or the **PR description** if you hit it on your own branch. For any bug that cost you more than an hour, and any you don't fix yet.

Stuck?

- **Time-box it** (an hour), then change something: explain it aloud, step away, ask.
- **AI's answer is hypothesis #4.** Test it like the others; an unchecked explanation is a confident guess.

Not fixing it yet?

It's a **known issue**: what happens, when it bites, any workaround, the issue link, in `docs/known-issues.md`.

Don't: change several things at once · stop at "it works now" without knowing why · fix before you understand.

CUT HERE

CS 301R · KEEP IT BESIDE THE KEYBOARD

The Debugging Loop

Method beats knowledge: evidence before belief.

0. **Say the failure exactly, and reproduce it.** Expected vs. actual, with the details ("*every* row, *the same* date"). Find the steps that make it happen every time.
1. **Hypothesize: two or three, not one.** Each names *where* the defect could be: the data, the display, the query, the config, the input.
2. **Test the cheapest one first,** not the one you believe. Vary the input · look at the stored data · add one log line where two guesses split · the debugger · `git log` / `git diff` / `git bisect` · shrink the case.
3. **Review.** Dead? Cross it out and say so: that's progress. Standing? Narrow until you can **point to the defect**, **explain when it bites**, and **propose a fix**.
4. **Fix and verify.** A failing test first (and ask why the old tests missed it) · fix the defect, not the symptom · run everything · walk the feature: did the fix break it? · remove your log lines.

You see the **failure** → the program hits a **fault** → caused by a **defect** in the code → from an **error**. Investigate backward.

The debug log: three lines

Saw	the failure, precisely
Suspected	each hypothesis, wrong ones too, and the evidence that settled it
Was	the defect, the fix, the test that now covers it

In the **bug's issue**, or the **PR description** if you hit it on your own branch. For any bug that cost you more than an hour, and any you don't fix yet.

Stuck?

- **Time-box it** (an hour), then change something: explain it aloud, step away, ask.
- **AI's answer is hypothesis #4.** Test it like the others; an unchecked explanation is a confident guess.

Not fixing it yet?

It's a **known issue**: what happens, when it bites, any workaround, the issue link, in `docs/known-issues.md`.

Don't: change several things at once · stop at "it works now" without knowing why · fix before you understand.

CS 301R